SIMATS Engineering

Python Programming - Assignment 1

Design, Implement and Validate a Payment Calculator
for a Village Milk Co-operative

Name	Swetha N S
Register number	1925260105A
Course	CSA08 - Python Programming (CO1)
Python version	Python 3.12.3
Libraries used	Standard library only: decimal, math, sys, time, random
GitHub repository	https://github.com/swethasundar1108/CSA0801A

How to run any program: `cd TaskNN` then `python3 <script>.py`. Task 9 tests: `cd Task09; python3 test_calculator.py`.

Task	Folder / script	Summary
1	Task01/task01_data.py	Inputs, outputs, types, constants, validation, assumptions
2	Task02/task02.py	Types, arithmetic, float vs Decimal vs paise
3	Task03/constants_demo.py	Named constants, dynamic typing, swapping
4	Task04/expressions.py	Steps, precedence, round/int/floor
5	Task05/input_demo.py	input(), conversion, try/except, f-strings
6	Task06/functions.py	rate_per_litre, payout, ten_day_total
7	Task07/milkrate.py, use_milkrate.py	Module and import
8	Task08/README.md	Pseudocode, flowchart, hand-trace
9	Task09/milk_calc.py, test_calculator.py	Final Decimal calculator and test table
10	Task10/experiments.py	float vs Decimal, timing, rate table
11	Task11/README.md	Final design recommendation
12	Task12/seasonal.py, original_vs_new.py	Seasonal rates and bonus

Task 1 - Understand the Problem and Identify the Data

Source: Task01/task01_data.py

```
"""Task1: data definition - inputs, named constants, types and validation rules."""
from decimal import Decimal

# Named constants (Decimal so money and percentages are exact)
BASE_RATE = Decimal("30.00")        # Rs per litre at 3.5% fat and 8.5% SNF
STD_FAT = Decimal("3.5")
STD_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")    # Rs per litre per 0.1% fat
SNF_STEP_VALUE = Decimal("0.30")    # Rs per litre per 0.1% SNF
STEP = Decimal("0.1")
MIN_FAT,  MAX_FAT  =  Decimal("2.0"),  Decimal("7.0")
MIN_SNF,  MAX_SNF  =  Decimal("6.0"),  Decimal("10.0") # assumption
VALID_SHIFTS = ("M", "E")

# One sample entry showing the chosen data types
entry = {
    "farmer_id":      "F0123",   # str  (identifier, not used in arithmetic)
    "litres":   Decimal("8.5"),   # Decimal
    "fat":      Decimal("4.2"),   # Decimal
    "snf":      Decimal("8.7"),   # Decimal
    "shift": "M",                 # str
}

def is_valid(e):
    """Return True when the entry satisfies every validation rule."""
    return (bool(e["farmer_id"])
    and e["litres"] >= 0
    and MIN_FAT <= e["fat"] <= MAX_FAT
    and MIN_SNF <= e["snf"] <= MAX_SNF
    and e["shift"] in VALID_SHIFTS)

if __name__ == "__main__":
    for key, value in entry.items():
        print(f"{key:10} {value!r:12} {type(value).__name__}")
    print("valid entry          :", is_valid(entry))
    print("negative litres      :", is_valid({**entry, "litres": Decimal("-1")}))
    print("fat 7.5 (too high)   :", is_valid({**entry, "fat": Decimal("7.5")}))
    print("shift 'X'            :", is_valid({**entry, "shift": "X"}))
```

Output: python3 Task01/task01_data.py

```
farmer_id  'F0123'      str
litres     Decimal('8.5')  Decimal
fat        Decimal('4.2')  Decimal
snf        Decimal('8.7') Decimal
shift      'M'       str   :
valid         entry  True  :
negative litres fat  False :
7.5    (too    high)  False :
shift 'X'            False
```

1.1 Inputs

Input	Meaning	Python type	Example
farmer_id	Identifier of the farmer; an identifier, not used in arithmetic (leading zeros possible)	str	'F0123'
litres	Quantity of milk; fractional values are possible	Decimal (or float)	8.5
fat	Fat percentage	Decimal (or float)	4.2
snf	Solids-not-fat percentage	Decimal (or float)	8.7
shift	Morning or evening collection	str ('M' / 'E')	'M'
is_valid	Result of validation	bool	True
entry_count	Number of entries for a farmer	int	20

1.2 Outputs

- **rate_per_litre** - price for the milk quality (Decimal, 2 places).
- **payment_amount** - litres x rate (Decimal, rounded to paise).
- **ten_day_total** - sum of the payments of one farmer in the ten-day period (Decimal).

1.3 Named constants

Constant	Value	Type
BASE_RATE	30.00 (Rs per litre)	Decimal
STD_FAT	3.5 (%)	Decimal
STD_SNF	8.5 (%)	Decimal
FAT_STEP_VALUE	0.40 per litre per 0.1% fat	Decimal
SNF_STEP_VALUE	0.30 per litre per 0.1% SNF	Decimal
STEP	0.1 (% size of one step)	Decimal
MIN_FAT, MAX_FAT	2.0 and 7.0 (%)	Decimal
MIN_SNF, MAX_SNF	6.0 and 10.0 (%) - assumed	Decimal
PAISE	0.01 (rounding quantum)	Decimal

1.4 Validation rules

- litres must be a number and **>= 0**.
- fat must be a number with **2.0 <= fat <= 7.0**.
- snf must be a number within the assumed range **6.0 to 10.0**.
- Text such as 'eight' or '8,5' is rejected (ValueError); 'nan' and 'inf' are rejected.
- farmer_id must be a non-empty string; shift must be 'M' or 'E'.

1.5 Assumptions and justification

- Rate adjustments are applied per **whole 0.1% step** (the readings from the clerk have one decimal); the step count is rounded to the nearest step.
- The brief gives limits only for fat, so an SNF range of 6.0-10.0% is assumed to catch typing mistakes.
- Rate revisions happen occasionally, so the rates are stored in one place (constants / rate card).
- Money is rounded to paise (0.01) with half-up rounding, as is usual for payments.
- Each farmer has about 20 entries per ten-day period (two shifts x ten days).

1.6 Additional question: separate variables vs list/tuple

Separate variables (litres1, fat1, snf1, ... litres300, ...) need 900 names and a separate line of code for every farmer. **A list of tuples** holds all entries in one object and is processed with one loop.

	Separate variables	List of tuples
Time to process n entries	O(n) - but each entry needs its own statement written in the code; cannot loop over names	O(n) - one loop, indexing is O(1)
Space	O(n) names in the namespace plus O(n) lines of source	O(n) objects; the code size stays O(1)
Adding farmers (300 to 6,000 to 150,000)	Code must be rewritten; unrealistic	No code change; data can be read from a file
Maintainability	Very poor, easy to mix up fat17 / fat71	Good

Both are O(n) in time and space for the data, but the list/tuple approach is far more **scalable** because the program size does not grow with the number of farmers.

Task 2 - Explore Values and Types in the Python Interpreter

Annotated interactive session (reproduced by `Task02/task02.py`):

```
>>> type(10), type(10.0), type("10"), type(True), type([10, 8.5])
(<class 'int'>, <class 'float'>, <class 'str'>, <class 'bool'>, <class 'list'>)
>>> 7 / 2      # true division, always float
3.5
>>> 7 // 2     # floor division
3
>>> 7 % 2      # remainder
1
>>> -7 // 2    # floors towards minus infinity, not towards zero
-4
>>> 0.1 + 0.2 == 0.3
False
>>> 0.1 + 0.2
0.30000000000000004
```

Why 0.1 + 0.2 == 0.3 is False: floats are stored in binary (base 2). 0.1, 0.2 and 0.3 have no exact binary representation, so each is stored as the nearest binary fraction. The sum of the two stored values is slightly different from the stored value of 0.3.

Why this matters for money: the tiny errors add up over thousands of additions (and in comparisons they make equality checks fail). A co-operative paying 300 to 150,000 farmers cannot accept totals that differ by paise from the bank amount.

Approach	Idea	Strength	Weakness
Integer paise	Store 33.40 as 3340 (int)	Exact, fastest, small	Manual conversion; division / percentages need care
round()	Round floats to 2 places	Simple	Hides but does not remove binary error; uses round-half-even; round(2.675, 2) gives 2.67
decimal module	Decimal('33.40') exact base-10	Exact, choice of rounding mode, readable	Slower and uses more memory than float

Completed code (TODO 1)

```
def total_decimal(items):
    """Add the amounts exactly using Decimal built from the strings."""
    total = Decimal("0")
    for amount in items:
        total += Decimal(amount)
    return total
```

Output: python3 Task02/task02.py

```
float : 33400.30000000061
Decimal: 33400.30
False True
Decimal(float(amount)): 33400.2999999999857893118278
integer paise: (33400.3, 3340030)
float + round each add: 33400.3
getsizeof float : 24
getsizeof int : 28
getsizeof Decimal: 104
10 <class 'int'>
10.0 <class 'float'>
'10' <class 'str'>
True <class 'bool'>
[10, 8.5] <class 'list'>
3.5 3 1 -4
```

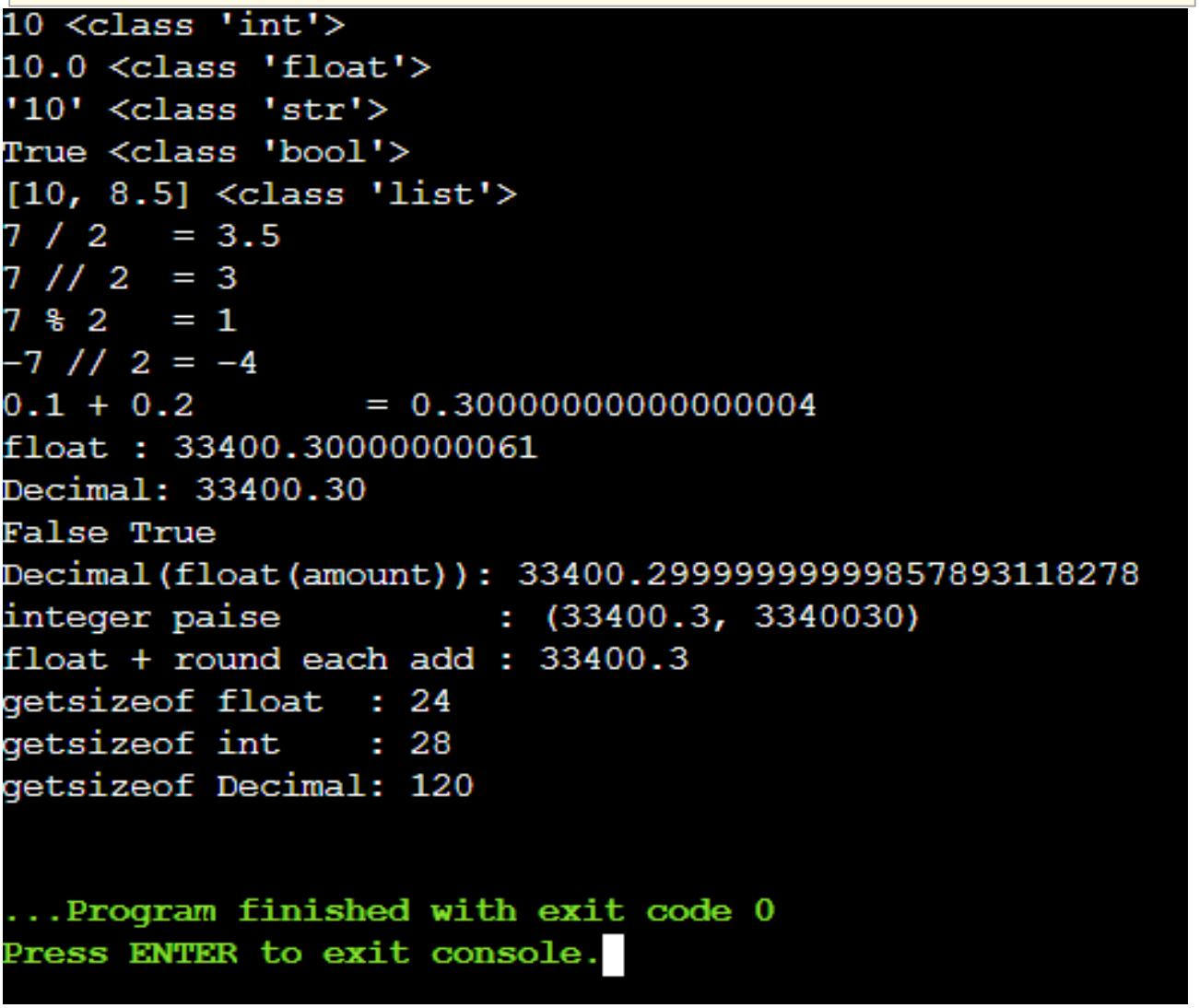
Expected output matched: `float : 33400.30000000061, Decimal: 33400.30, False True.`

Decimal(float(amount)): the result becomes 33400.29999999999857.... float('33.40') has already been converted to the nearest binary number, which is not exactly 33.40; Decimal(float) then faithfully copies that inexact binary value. Always build Decimal from a **string**.

Additional question: Decimal vs integer paise vs float with rounding

Method	Implementation	Time per addition	Memory (sys.getsizeof)	Exact?
Decimal	total += Decimal(s)	O(1) for fixed precision (slower constant)	Decimal: 104 bytes	Yes
Integer paise	paise += round(float(s)*100)	O(1), fastest	int: 28 bytes	Yes (once the value is converted to paise)
float + round after each add	total = round(total + x, 2)	O(1)	float: 24 bytes	Not guaranteed - it gave 33400.3 here, but it relies on rounding binary values and can fail for other data

[the Task 2 session and output]



Task 3 - Choose Variables and Name the Constants

Source: Task03/constants_demo.py

```
BASE_RATE = 30.00
STD_FAT = 3.5
STD_SNF = 8.5
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STEP_SIZE = 0.1
MIN_FAT, MAX_FAT = 2.0, 7.0
RATE_CARD = {"base": 30.0, "fat": 0.40, "snf": 0.30}



x = 10; print(type(x)); x = "ten"; print(type(x))   # dynamic typing
a, b = 3, 5; a, b = b, a; print(a, b)               # swap
c = d = 0; e, f = 1, 2; print(c, d, e, f)           # multiple assignment
BASE_RATE = 99.0   # Python allows this: constants are convention only
print(BASE_RATE)
```

Output: python3 Task03/constants_demo.py

```
<class 'int'>
<class 'str'>
5 3
0 0 1 2
99.0
```

- **Naming:** constants use UPPER_CASE (PEP 8); ordinary variables use lower_snake_case.
- **Python does not enforce constants.** The last lines of the program rebind BASE_RATE to 99.0 without any error. Python relies on the convention: a programmer seeing UPPER_CASE knows the value must not be reassigned. Tools (linters, type checkers) can warn about it.
- **Dynamic typing:** the same name x held an int and then a str - the type belongs to the value, not the variable.
- **Multiple assignment / swap:** a, b = b, a builds a tuple of the right-hand values and then unpacks, so no temporary variable is needed.

Additional question: magic numbers vs constants vs dictionary

	Magic numbers	Named constants	RATE_CARD dictionary
Effort to revise a rate	Find every occurrence	Edit one line	Edit one entry (or load another dictionary)
Places to modify	Many (every expression using it); easy to miss one	1	1
Readability	Poor: what is 0.4?	Good: FAT_STEP_VALUE	Good: RATE_CARD['fat']
Extra benefit	-	Simple, fast	Easy to select a card by season or load from a file
Time / memory	O(1), O(1)	O(1) per use, O(1)	O(1) average (hash lookup, a few ns slower), O(1) for a 3-entry dict

The three options do **not** change running time or memory significantly - the difference is in maintenance. I chose named constants for the base design and a dictionary-based rate card when seasons were introduced (Task 12).

Task 4 - Write Correct Expressions

```
fat_steps  = round((fat - 3.5) / 0.1)        # steps of 0.1% relative to 3.5%
snf_steps  = round((snf - 8.5) / 0.1)        # steps of 0.1% relative to 8.5%
rate       = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
payment    = round(litres * rate, 2)
```

Source: Task04/expressions.py

```
import math
print("(4.2-3.5)/0.1 =", (4.2 - 3.5) / 0.1, "-> round:", round((4.2 - 3.5) / 0.1))
print("(3.1-3.5)/0.1 =", (3.1 - 3.5) / 0.1)
print("(8.7-8.5)/0.1 =", (8.7 - 8.5) / 0.1)
print("-2**2 =", -2 ** 2, " (-2)**2 =", (-2) ** 2, " 2**3**2 =", 2 ** 3 ** 2)
print("negatives: round(-2.5)=", round(-2.5), "int(-2.5)=", int(-2.5), "floor(-2.5)=", math.floor(-2.5))
BASE, FATV, SNFV = 30.0, 0.40, 0.30
for fat, snf in [(4.2, 8.5), (3.1, 8.5)]:
    raw = (fat - 3.5) / 0.1
    for name, fn in [("round", round), ("int", int), ("floor", math.floor)]:
        steps = fn(raw)
        print(f"fat {fat}: raw={raw!r:20} {name:5} steps={steps:3} rate={BASE + steps*FATV:.2f}")
fat = 4.2
print(2.0 <= fat <= 7.0, 2.0 <= fat and fat <= 7.0, not (fat < 2.0 or fat > 7.0))
```

Output: python3 Task04/expressions.py

```
(4.2-3.5)/0.1 = 7.000000000000002 -> round: 7
(3.1-3.5)/0.1 = -3.999999999999999
(8.7-8.5)/0.1 = 1.999999999999993
-2**2 = -4  (-2)**2 = 4  2**3**2 = 512
negatives: round(-2.5)= -2 int(-2.5)= -2 floor(-2.5)= -3
fat  4.2:  raw=7.000000000000002   round steps= 7 rate=32.80
fat  4.2:  raw=7.000000000000002 int     steps= 7 rate=32.80
fat  4.2:  raw=7.000000000000002   floor steps= 7 rate=32.80
fat  3.1:  raw=-3.999999999999999  round steps= -4 rate=28.40
fat  3.1:  raw=-3.999999999999999 int    steps= -3 rate=28.80
fat  3.1:  raw=-3.999999999999999  floor steps= -4 rate=28.40
True True True
```

Explanations

- **Precedence (high to low):** ** , unary + -, * / // %, binary + -, comparisons, not, and, or. Operators of the same level group left to right (**left-associative**), but ** is **right-associative**: 2**3**2 = 2**(3**2) = 512.
- **Unary minus vs **:** ** binds tighter than unary minus on its left, so -2**2 = -(2**2) = -4, while (-2)**2 = 4.
- **Comparison chaining:** 2.0 <= fat <= 7.0 means (2.0 <= fat) and (fat <= 7.0), with fat evaluated once. Boolean operators and / or / not combine conditions and short-circuit.
- **Three unexpected results observed:** (1) (4.2-3.5)/0.1 = 7.000000000000002; (2) (3.1-3.5)/0.1 = -3.999999999999999; (3) (8.7-8.5)/0.1 = 1.999999999999993. (The exact tail depends on the numbers; the result is near, not equal to, the integer.) A fourth surprise: round(-2.5) = -2 because Python rounds halves to the even number.
- **round(), int(), math.floor() for negatives:** round(-2.5) = -2 (nearest, half to even); int(-2.5) = -2 (truncates toward zero); floor(-2.5) = -3 (toward minus infinity).

Additional question: round vs int vs floor for fat 4.2 and 3.1

Fat	Raw value	Method	Steps	Rate (Rs/L)	Correct?
4.2	7.000000000000002	round / int / floor	7 / 7 / 7	32.80	All correct (tail is above 7) Correct
3.1	-3.999999999999999	round	-4	28.40	**Wrong**: truncates toward zero
3.1	-3.999999999999999	int	-3	28.80	Correct here, but fails if the error is on the other side (e.g. 7.0 stored as 6.999999999999999 gives 6)
3.1	-3.999999999999999	floor	-4	28.40	

Complexity: all three are O(1) time and O(1) space per entry. **round()** is the right choice because it recovers the intended whole number from a value that is only close to it.

Task 5 - Read Input and Convert Types

- **Why input() returns a string:** the keyboard (standard input) delivers characters; Python cannot know whether '10' is a number, an ID or a name. The program decides the type by converting explicitly.
- **Conversion:** int('12') gives 12, float('8.5') gives 8.5. int('8.5') raises ValueError.
- **Invalid values:** float('8,5') and float('eight') raise ValueError. A program must catch it rather than crash.
- **Validation:** either check the text first (string methods) or convert inside try/except; then check the range (for example 2.0 <= fat <= 7.0).

Source: Task05/input_demo.py

```
def parse_float(text):
    try:
        return float(text)
    except ValueError:
        return None

def parse_with_string_checks(text):
    t = text.strip()
    if t.replace(".", "", 1).isdigit():
        return float(t)
    return None

for s in ["8.5", "8,5", " 8.5 ", "eight"]:
    print(f"{s!r:8} try/except -> {parse_float(s)!s:5} | string checks -> {parse_with_string_checks(s)}")
amount = 283.9
print(f"[{amount:8.2f}] [{'F001':>6}] [{12:5d}]")

def read_float(prompt):          # interactive version
    while True:
        value = parse_float(input(prompt))
        if value is not None:
            return value
        print("Invalid number, try again.")
```

Output: python3 Task05/input_demo.py

```
'8.5'    try/except -> 8.5   | string checks -> 8.5
'8,5'    try/except -> None  | string checks -> None
' 8.5 '  try/except -> 8.5   | string checks -> 8.5
'eight'  try/except -> None  | string checks -> None
[  283.90] [  F001] [   12]
```

f-string formatting: `{amount:8.2f}` means width 8 and 2 decimals; `{'F001':>6}` right-aligns text in width 6.

Additional question: string-method validation vs try/except

Input	String methods (after strip())	try / except ValueError
'8,5'	Rejected: the comma is not a digit	float raises ValueError: rejected
' 8.5 '	Rejected unless strip() is called first	Accepted: float() ignores surrounding spaces
'eight'	Rejected	ValueError: rejected
Other cases	isdigit() fails for '-8.5' and '1e3'; many special cases to code	Accepts '-8.5' and '1e3'; also accepts 'nan' and 'inf', so a finite-number check is still needed

Time: both are O(L) per entry, where L is the length of the text. The string approach scans the text several times (replace, isdigit, ...) so it has a larger constant; try/except is a single pass and is the more Pythonic ('easier to ask forgiveness than permission'). In the final design I use try/except and then range validation.

[The input program with valid and invalid input]

```
type of what input() returns: <class 'str'>
int('12')    -> 12
float('8.5')-> 8.5
float('8,5') raised ValueError
float('eight') raised ValueError

input      try/except    string methods
'8.5'      8.5           8.5
'8,5'      None          None
' 8.5 '    8.5           8.5
'eight'    None          None
'-8.5'     -8.5          None

ID       Litres    Fat    SNF     Rate     Amount   Status
F001       10.0    3.5    8.5    30.00     300.00   OK
F002        8.5    4.2    8.7    33.40     283.90   OK
F003    invalid number entered
F004    value out of range


...Program finished with exit code 0
Press ENTER to exit console.
```

Task 6 - Organise the Logic into Functions

Source: Task06/functions.py

```
BASE_RATE, FAT_STEP_VALUE,SNF_STEP_VALUE = 30.00, 0.40, 0.30

def rate_per_litre(fat, snf):
    """Return price per litre for the given fat and SNF percentage."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return payment amount for one delivery."""
    return round(litres * rate_per_litre(fat, snf), 2)

def ten_day_total(entries):
    """Return total payment for a farmer; entries = [(litres, fat, snf), ...]."""
    return round(sum(payout(l, f, s) for l, f, s in entries), 2)

if __name__ == "__main__":
    week = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    print(ten_day_total(week))
    square = lambda x: x * x
    print(square(4))
```

Output: python3 Task06/functions.py

```
917.5
16
```

- **Parameters / return values:** rate_per_litre(fat, snf) returns a number; payout(litres, fat, snf) returns the amount; ten_day_total(entries) takes a list of tuples and returns the sum.
- **Return vs print:** return hands a value back to the caller so it can be stored, tested, added up or formatted; print only displays text and the function then returns None. Calculation functions should return, and the main program should print.
- **Local vs global:** variables created inside a function (fat_steps, snf_steps) are local and disappear when it returns. Names defined at module level (BASE_RATE ...) are global.
- **Constants inside functions:** a function can read a global without any declaration. Assigning to it would create a new local variable (or need the global keyword and change the rate for everyone), so rates should be changed only at their definition.

Additional question: one script vs functions vs lambda

	One long script	Reusable functions	Lambda
Reuse	Copy and paste	Call anywhere / import	Only inside one expression
Repeated logic	Same formula repeated for every use	Written once	Written once but anonymous
Maintainability	Poor: fix in many places	Good: one fix, docstrings, testable	Good for tiny one-liners (sort keys); no docstring, harder to debug
Call overhead for n entries	None	About n calls, a small constant (fractions of a microsecond) each; total O(n) as before	Same as a function

Function-call overhead changes only the constant factor; the complexity is O(n) in all three. The clarity and testability of functions outweigh the small cost.

Task 7 - Build and Use a Python Module

Completed milkrate.py (TODO 1 and TODO 2)

Source: Task07/milkrate.py

```
"""Rate card of thevillage milk co-operative."""

BASE_RATE    =   30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Return the price per litre, rounded to two decimals."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return the payment amount rounded to two decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf}" f" rate {rate:6.2f} pay {pay:8.2f}")
```

Output: python3 milkrate.py (run directly) - matches the expected output

```
   10 L fat 3.5 snf 8.5 rate  30.00 pay   300.00
  8.5 L fat 4.2 snf 8.7 rate  33.40 pay   283.90
   12 L fat 3.1 snf 8.3 rate  27.80 pay   333.60
```

Source: Task07/use_milkrate.py

```
import sys
import milkrate as mr
print("Payout for 20 L, 4.0 fat, 8.6 SNF:", mr.payout(20, 4.0, 8.6))
print("milkrate cached in sys.modules:", "milkrate" in sys.modules)
```

Output: python3 use_milkrate.py (imports the module)

```
Payout for 20 L, 4.0 fat,8.6 SNF:646.0
milkrate cached in sys.modules: True
```

- **import milkrate** - loads the module; functions are used as milkrate.payout(...). **from milkrate import payout** - copies only the name payout into the current namespace, called as payout(...); risks name clashes. **import milkrate as mr** - same as the first form with a short alias (mr.payout).
- **if __name__ == "__main__":** when a file is run directly, __name__ is '__main__'; when it is imported, __name__ is 'milkrate'. The demonstration table is inside this block, so it is **not printed on import** - importing the module only defines the functions.
- If the demonstration code were not protected, every script that imports the module would print the table (unwanted output, slower start-up, possible side effects such as reading input).
- **Standard modules:** math (floor, isclose, ...) and decimal (exact decimal arithmetic) ship with Python and are imported the same way.
- **Search order:** Python looks in sys.modules (cache), then in the directory of the running script, then PYTHONPATH, then the standard library and site-packages; the list is sys.path.

Additional question: module vs copying functions

	Import shared module	Copy functions into every script
Rate revision	Edit milkrate.py once	Edit m scripts; easy to forget one, producing different payments
Maintainability	Single tested source	Duplicated bugs
Cost	One-time cost to find, compile and run the module: O(size of module). Later imports are an O(1) dictionary lookup in sys.modules	No import cost but O(m) maintenance and extra source space

Python stores each imported module object in the dictionary sys.modules; a second `import milkrate` finds it there and does not execute the file again (the output above shows True).

Task 8 - Plan the Algorithm Before Coding

Stages of algorithmic problem-solving: (1) **Understand** - identify inputs, outputs, rules and constraints; (2) **Plan** - write pseudocode and a flowchart and trace sample data; (3) **Write** - translate the plan into Python; (4) **Test** - run normal, boundary and invalid cases and compare with the hand-trace.

Pseudocode - one payment

```
ALGORITHM CalculatePayment
```

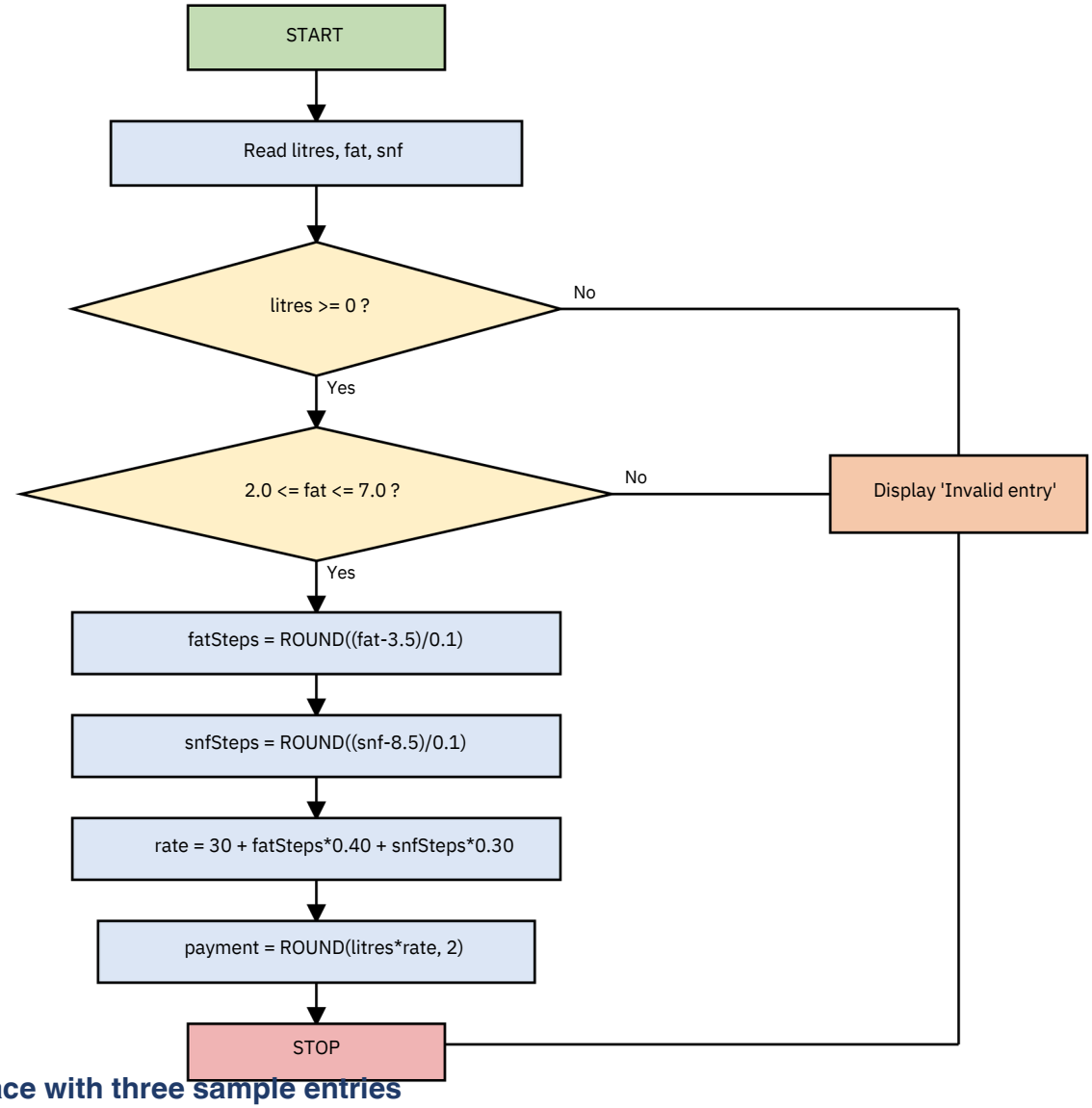
```
INPUT   litres, fat, snf
OUTPUT rate, payment
CONSTANTS BASE_RATE = 30.00, FAT_STEP_VALUE = 0.40, SNF_STEP_VALUE = 0.30,
          STD_FAT = 3.5, STD_SNF = 8.5, STEP = 0.1, MIN_FAT = 2.0, MAX_FAT = 7.0
BEGIN
  IF litres < 0 OR fat < MIN_FAT OR fat > MAX_FAT THEN
      DISPLAY "Invalid entry"
      RETURN
  END IF
  fatSteps <- ROUND((fat - STD_FAT) / STEP)
  snfSteps <- ROUND((snf - STD_SNF) / STEP)
  rate      <- BASE_RATE + fatSteps * FAT_STEP_VALUE + snfSteps * SNF_STEP_VALUE
  payment   <- ROUND(litres * rate, 2)
  RETURN rate, payment
END
```

Pseudocode - ten-day total for one farmer

```
ALGORITHM TenDayTotal
```

```
INPUT   entries (list of litres, fat, snf)
OUTPUT total
BEGIN
  total <- 0
  FOR EACH entry IN entries DO
      IF entry is valid THEN
          total <- total + CalculatePayment(entry.litres, entry.fat, entry.snf).payment
      ELSE
          DISPLAY "Entry skipped"
      END IF
  END FOR
  RETURN total
END
```

Flowchart - validation and calculation of one entry



Hand-trace with three sample entries

Entry	litres	fat	snf	valid?	fatSteps	snfSteps	rate	payment	running total
1	10	3.5	8.5	Yes	0	0	30.00	300.00	300.00
2	8.5	4.2	8.7	Yes	7	2	30+2.80+0.60 = 33.40	283.90	583.90
3	12	3.1	8.3	Yes	-4	-2	30-1.60-0.60 = 27.80	333.60	917.50

The trace agrees with the program: payments 300.00, 283.90, 333.60 (Task 7) and a ten-day total of 917.50 (Task 6).

Additional question: store all entries vs running total

	Store every entry in a list	Running total
Time for n entries	O(n)	O(n)
Space	O(n)	O(1)
When necessary	Receipts and audit trail, re-calculation after a rate correction, per-shift reports, sorting and statistics, bonus based on total litres (Task 12)	Only the final total is needed and entries can be streamed from a file

Task 9 - Implement and Test the Calculator

The final calculator uses Decimal for all money and percentages, named constants, try/except conversion, range validation and small functions.

Source: Task09/milk_calc.py

```
"""Final calculator: Decimal arithmetic, validation, functions."""
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STD_FAT, STD_SNF, STEP = Decimal("3.5"), Decimal("8.5"), Decimal("0.1")
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
MIN_SNF, MAX_SNF = Decimal("6.0"), Decimal("10.0")   # assumption
PAISE = Decimal("0.01")

def money(x):
    return x.quantize(PAISE, rounding=ROUND_HALF_UP)

def parse_decimal(text):
    """Convert text to Decimal; raise ValueError for bad input (e.g. '8,5', 'eight')."""
    try:
        value = Decimal(text.strip())
    except Exception:
        raise ValueError(f"not a number: {text!r}")
    if not value.is_finite():
        raise ValueError(f"not a finite number: {text!r}")
    return value

def validate(litres, fat, snf):
    if litres < 0:
        raise ValueError("litres must be non-negative")
    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("fat must be between 2.0 and 7.0")
    if not MIN_SNF <= snf <= MAX_SNF:
        raise ValueError("SNF must be between 6.0 and 10.0")

def steps(value, standard):
    return ((value - standard) / STEP).to_integral_value(rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    return money(BASE_RATE + steps(fat, STD_FAT) * FAT_STEP_VALUE
                 + steps(snf, STD_SNF) * SNF_STEP_VALUE)

def payout(litres, fat, snf):
    return money(litres * rate_per_litre(fat, snf))

def ten_day_total(entries):
    return sum((payout(l, f, s) for l, f, s in entries), Decimal("0.00"))

def process(litres_t, fat_t, snf_t):
    l, f, s = parse_decimal(litres_t), parse_decimal(fat_t), parse_decimal(snf_t)
    validate(l, f, s)
    return payout(l, f, s)
```

Source: Task09/test_calculator.py

```
from decimal import Decimal as D
from milk_calc import *

cases = [
 ("above standard", ("8.5","4.2","8.7"), D("283.90")),
 ("below standard", ("12","3.1","8.3"), D("333.60")),
("boundary 3.5/8.5", ("10","3.5","8.5"), D("300.00")),
("lowest fat 2.0", ("10","2.0","8.5"), D("240.00")),
("highest fat 7.0", ("10","7.0","8.5"), D("440.00")),
("invalid text", ("10","eight","8.5"), ValueError),
 ("comma decimal", ("10","3,5","8.5"), ValueError),
 ("negative litres", ("-5","3.5","8.5"), ValueError),
 ("fat too high", ("10","7.1","8.5"), ValueError),
 ("fat too low", ("10","1.9","8.5"), ValueError),
]
print(f"{'Case':18}{'Input':26}{'Expected':>12}{'Actual':>12}  Result")
for name, inp, exp in cases:
    try:
        act = process(*inp)
    except ValueError:
        act = ValueError
    ok = act == exp
    exp_s = getattr(exp, "__name__", str(exp))
    act_s = getattr(act, "__name__", str(act))
    status = "PASS" if ok else "FAIL" print(f"{name:18}{str(inp):26}
    {exp_s:>12}{act_s:>12} {status}") assert ok


rows = [("10", "3.5", "8.5"), ("8.5", "4.2", "8.7"), ("12", "3.1", "8.3"),
("20", "4.0", "8.6"), ("5", "3.6", "8.4")] entries
= [(D(a), D(b), D(c)) for a, b, c in rows] manual =
(D("300.00") + D("283.90") + D("333.60")
          + D("20") * D("32.30") + D("5") * D("30.10"))
print("5-entry total actual:", ten_day_total(entries), "manual:", manual)
assert ten_day_total(entries) == manual
print("ALL TESTS PASSED")
```

Test table (actual program output; every case passes)

```
Case              Input                          Expected      Actual  Result
above standard    ('8.5', '4.2', '8.7')            283.90      283.90  PASS
below standard    ('12', '3.1', '8.3')             333.60      333.60  PASS
boundary 3.5/8.5 ('10', '3.5', '8.5')              300.00      300.00  PASS
lowest fat 2.0    ('10', '2.0', '8.5')             240.00      240.00  PASS
highest fat 7.0   ('10', '7.0', '8.5')             440.00      440.00  PASS
invalid text      ('10', 'eight', '8.5')         ValueError  ValueError  PASS
comma decimal     ('10', '3,5', '8.5')           ValueError  ValueError  PASS
negative litres   ('-5', '3.5', '8.5')           ValueError  ValueError  PASS
fat too high      ('10', '7.1', '8.5')           ValueError  ValueError  PASS
fat too low       ('10', '1.9', '8.5')           ValueError  ValueError  PASS
5-entry total actual: 1714.00 manual: 1714.00
ALL TESTS PASSED
```

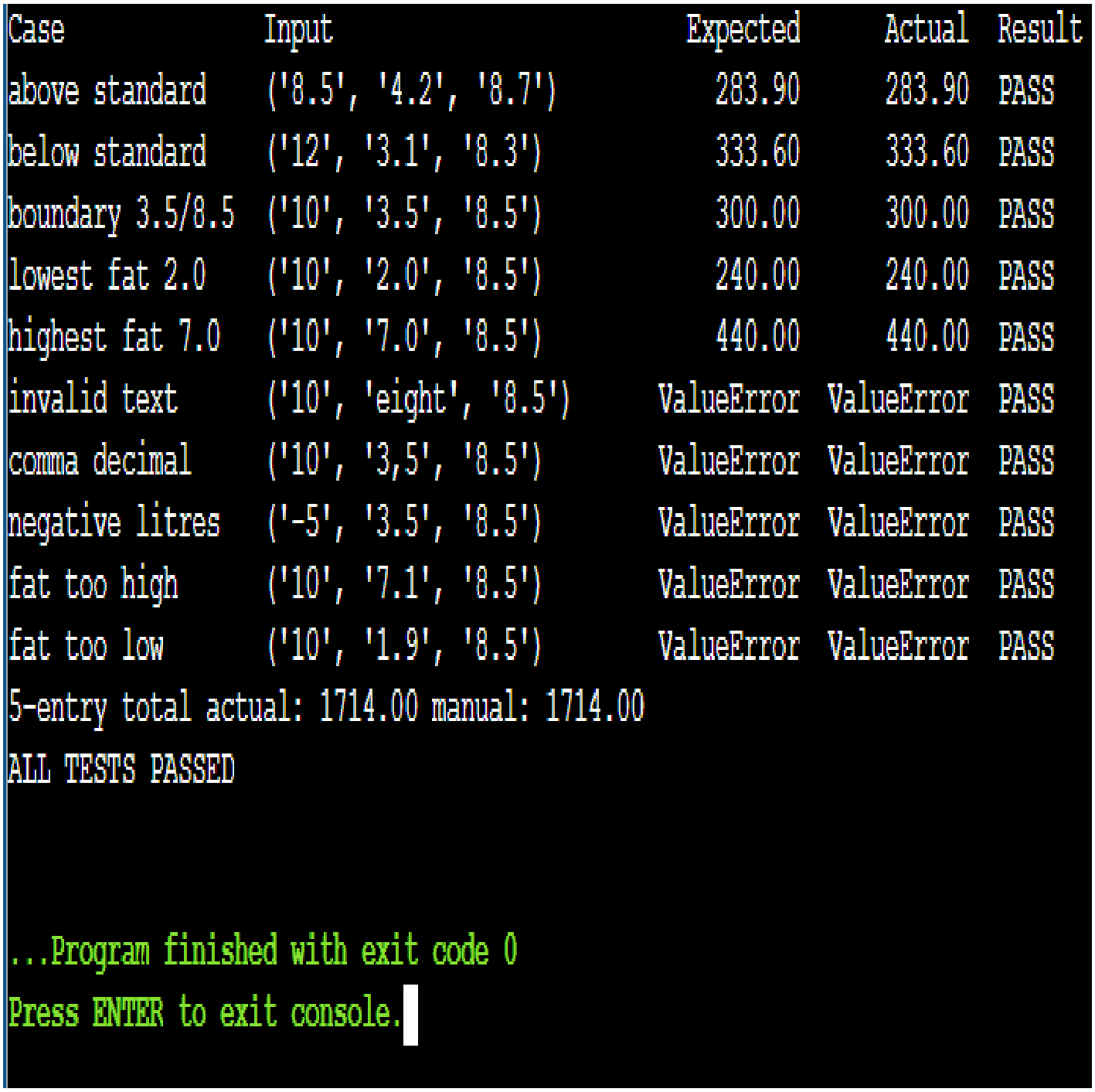
Test	Input (litres, fat, snf)	Expected	Actual	Result
Above standard	8.5, 4.2, 8.7	283.90	283.90	PASS
Below standard	12, 3.1, 8.3	333.60	333.60	PASS
Boundary 3.5 / 8.5	10, 3.5, 8.5	300.00	300.00	PASS
Lowest fat 2.0	10, 2.0, 8.5	240.00 (rate 24.00)	240.00	PASS
Highest fat 7.0	10, 7.0, 8.5	440.00 (rate 44.00)	440.00	PASS
Invalid text	10, eight, 8.5	ValueError	ValueError	PASS
Comma decimal	10, 3,5, 8.5	ValueError	ValueError	PASS
Negative litres	-5, 3.5, 8.5	ValueError	ValueError	PASS
Fat 7.1 (too high)	10, 7.1, 8.5	ValueError	ValueError	PASS
Fat 1.9 (too low)	10, 1.9, 8.5	ValueError 1714.00	ValueError	PASS
Five-entry set	see below	(manual)	1714.00	PASS

Manual calculation of the five-entry set: 300.00 + 283.90 + 333.60 + 20 x 32.30 (4.0 fat, 8.6 snf: 30 + 5x0.40 + 1x0.30) = 646.00 + 5 x 30.10 (3.6 fat, 8.4 snf: 30 + 0.40 - 0.30) = 150.50, total **1714.00**.

Additional question: manual vs automated testing

Manual testing: the tester types each input, reads the output and compares it by eye.**Automated testing**: a function holds the cases and usesassert actual == expected; the whole suite runs with one command and fails loudly at the first wrong result. After a change (for example a rate revision), manual testing must be repeated from scratch - effort proportional to the number of cases T, every time, and error-prone - while automated testing costs the same one command and a few milliseconds of machine time (O(T)). For this reason I rerun test_calculator.py after every change. (Larger projects use unittest or pytest, which also report all failures.)

[python3 test_calculator.py output]



Task 10 - Compare Expectations with Execution

Source: Task10/experiments.py

```
import random, time
from decimal import Decimal, ROUND_HALF_UP
random.seed(1)
P = Decimal("0.01")


def gen(n):  # exact payments in 3 decimals, as products litres * rate can give
    return [Decimal(random.randint(10000, 60000)) / 1000 for _ in range(n)]
print(f"{'n':>8}{'float total':>22}{'Decimal total':>16}{'round each':>14}{'t_float(s)':>12}{'t_dec(s)':>10}")
for n in (300, 6000, 150000):
    data = gen(n); fl = [float(x) for x in data]
    t = time.perf_counter(); tf = sum(fl); t_f = time.perf_counter() - t
    t = time.perf_counter(); td = sum(data, Decimal(0)); t_d = time.perf_counter() - t
    each = sum((x.quantize(P, rounding=ROUND_HALF_UP) for x in data), Decimal(0))
    print(f"{n:>8}{tf:>22.10f}{td.quantize(P, rounding=ROUND_HALF_UP):>16}{each:>14}{t_f:>12.6f}{t_d:>10.6f}")
# rate-table vs recalculation
import sys; sys.path.insert(0, "../Task09")
from milk_calc import rate_per_litre
def random_pair():
    fat = Decimal(str(round(random.uniform(2, 7), 1)))
    snf = Decimal(str(round(random.uniform(7, 9.5), 1)))
    return fat, snf
pairs = [random_pair() for _ in range(150000)]
t = time.perf_counter(); [rate_per_litre(f, s) for f, s in pairs]; t_calc = time.perf_counter() - t
table = {}
t = time.perf_counter()
for p in pairs:
    if p not in table: table[p] = rate_per_litre(*p)
    _ = table[p]
t_tab = time.perf_counter() - t
print(f"recalculate 150000: {t_calc:.3f}s | dict lookup: {t_tab:.3f}s | distinct pairs k = {len(table)}")
```

Measured results (time.perf_counter); your own timings will differ a little

```
 n floattotal Decimaltotal roundeach t_float(s) t_dec(s)
 300 10623.2210000000 10623.22 10623.31 0.000002 0.000011   6000 209201.9020000000
209201.90  209204.88  0.000022  0.000157     150000  5247356.0340000000  5247356.03
5247430.82 0.000526 0.004003 recalculate 150000: 0.144s | dict lookup: 0.097s |
distinct pairs k = 1326
```

- **Test data:** synthetic payments with three decimals (seed 1) so that the rounding question is visible; real payments are products of litres and rates and also have more than two decimals.
- **float vs Decimal:** the totals agree to the paise in this experiment, but float carries a tiny binary error (as in Task 2, where 1,003 additions drifted to 33400.30000000061) with no guarantee; Decimal is exact. Decimal is roughly 5 to 10 times slower in summing but still only milliseconds for 150,000 payments.
- **Round each payment or only the final total?** The two methods differ: by Rs 0.09 for 300 payments, Rs 2.98 for 6,000 and Rs 74.79 for 150,000. A bank transfers each farmer an amount in whole paise, so the amount that actually leaves the account is the **sum of the individually rounded payments**. Rounding only the final total gives a figure that no set of real payments adds up to. Therefore the correct bank amount is obtained by rounding each payment.
- **Why they differ:** each rounding discards up to half a paisa; these discarded fractions accumulate. Half-up rounding of values ending in 5 in the third decimal tends to round upward, so the individually rounded sum is slightly higher.

Additional question: rate table (dictionary) vs recalculation

- **How it works:** key = (fat, snf); on the first occurrence the rate is calculated and stored in a dictionary; every later entry with the same pair is a hash lookup.
- **Time per entry:** lookup O(1) average, recalculation O(1) but with several Decimal operations (subtract, divide, round, multiply, add). In the measurement the dictionary version was faster (see output) because k = 1,326 distinct pairs are reused by 150,000 entries.
- **Space:** O(k) extra for k distinct pairs. Since fat and snf have one decimal, k is bounded (about 51 fat values x 41 SNF values = 2,091 here), so the table is small and the saving is worthwhile.

Task 11 - Recommend and Defend the Final Design

Recommendation

- **Data types:** Decimal for litres, fat, SNF, rates and amounts, always created from strings; int for counts; str for IDs and shifts.
- **Rounding strategy:** round each farmer's payment to paise with ROUND_HALF_UP, then add the rounded payments. This is justified by Task 10: it is the amount actually paid, and it differs from final-only rounding by Rs 74.79 at 150,000 payments.
- **Rate revision (safe procedure):** change the value in **one** place (constants or a rate card that carries an effective date); never overwrite past rates; rerun the automated tests (Task 9) with new expected values; compare old and new totals for a sample before releasing.
- **Structure:** module with small functions, validation separate from calculation, automated tests.

Limitations

- Everything is held in memory and entered as text - no database, no audit trail, no user interface.
- SNF limits are an assumption; the step rule ignores readings with more than one decimal until Task 12's pro-rata change.
- Decimal is slower than float and uses about 4 times more memory per number (104 vs 24 bytes).

Can it support 150,000 farmers?

Yes, with a streaming design. About 20 entries per farmer per ten days gives roughly 3 million entries. From Task 10, calculating 150,000 rates took about 0.1-0.2 s, so 3 million entries is about 2-4 s of calculation, which is acceptable for a payment run every ten days. Storing every entry as Decimal objects would need on the order of a gigabyte, so the program should read entries from a file and keep only a **running total per farmer** (O(1) memory per farmer, O(n) time); the rate table keeps the rate calculation cheap. For the block level (6,000 farmers, 120,000 entries) memory is not an issue at all.

Additional question: rate card in a JSON file

```
{ "base": "30.00", "fat_step": "0.40", "snf_step": "0.30" }
rates = json.load(open("rates.json"), parse_float=Decimal) # or keep values as strings
```

The program reads the file once at start-up and converts values to Decimal (store them as strings or use parse_float=Decimal; plain JSON floats would reintroduce binary error). **Start-up cost:** O(t) time and space for t rate values - microseconds to milliseconds for a handful of values. **Maintenance:** a rate revision is a text edit with no code change and no redistribution of the program, which is a big saving when many offices use it; the risks are typing errors or a missing file, so the loader must validate the values and fail clearly. Embedded constants are simpler and safer for a single small deployment; a JSON file is better when rates change often or the program runs at many centres.

Task 12 - Day-4 Design Challenge: Re-evaluate and Adapt the Design

A. Impact Analysis

-**What changed:** (1) analyser readings now have two decimals; (2) from April to June the base rate is Rs 32.00 and a fat step is worth Rs 0.45; (3) a bonus of Rs 0.50 per litre applies when a farmer supplies more than 500 litres in ten days. Before, the rates were fixed all year and there was no total-based rule.

-**Constants to modify:** BASE_RATE and FAT_STEP_VALUE are no longer single values - they depend on the season (RATE_CARDS). New constants: BONUS_LIMIT = 500, BONUS_PER_LITRE = 0.50, SUMMER_MONTHS = (4, 5, 6). SNF_STEP_VALUE is unchanged.

-**Expressions / functions to modify:** rate_per_litre and payout gain a season parameter; ten_day_total must add the bonus; new functions bonus() and season_for_month(). Validation must accept two decimals.

-**Effect of two-decimal readings:** with a step of 0.1% a reading such as 4.25 is 7.5 steps. The original round() then has to choose between 7 and 8, so readings 0.05 apart can pay the same, and halves are rounded to even (7.5 gives 8 but 8.5 gives 8). Floating-point noise makes it worse. The fix is to use exact Decimal arithmetic and let the step count be fractional (pro-rata).

B. Evaluation of the Original Design

Source: Task12/original_vs_new.py

```
"""Worked examples: original(Task7) design applied to two-decimal analyser readings."""
import sys
sys.path.insert(0, "../Task07")
import milkrate as old
import seasonal as new
from decimal import Decimal as D

print(f"{'fat':>5}{'snf':>6}{'raw fat steps':>22}{'old steps':>10}{'old rate':>10}{'new rate (normal)':>19}")
for fat, snf in [(4.25, 8.5), (4.35, 8.5), (4.45, 8.5), (4.15, 8.5), (3.55, 8.65)]:
    raw = (fat - 3.5) / 0.1
    print(f"{fat:>5}{snf:>6}{raw!r:>22}{round(raw):>10}{old.rate_per_litre(fat, snf):>10.2f}"
          f"{new.money(new.rate_per_litre(D(str(fat)), D(str(snf)))):>19}")
print("summer, original design cannot express: base 32.00 / fat step 0.45 / bonus")
```

Output: original design (Task 7) on two-decimal readings vs pro-rata Decimal calculation

```
  fat snf raw fat steps old steps old rate new rate (normal)
 4.25 8.5 7.5 8 33.20 33.00
 4.35 8.5 8.499999999999996 8 33.20 33.40
 4.45 8.5 9.500000000000002 10 34.00 33.80
 4.15 8.5 6.5000000000000036 7 32.80 32.60
 3.55 8.65 0.4999999999999982 0 30.60 30.65
summer, original design cannot express: base 32.00 / fat step 0.45 / bonus
```

- **Constants:** the three module constants cannot express two seasons - they would have to be edited twice a year, and past payments could not be reproduced.
- **Functions:** the structure (rate_per_litre, payout, ten_day_total) is still right, but the signatures lack the season and the bonus.
- **Worked example:** fat 4.35 gives raw steps 8.499999999999996, so round gives 8 steps (rate 33.20), while fat 4.45 gives 10 steps (rate 34.00) - a 0.10 increase in fat changes the rate by Rs 0.80, and 4.25 and 4.35 both pay 33.20. The exact pro-rata value is 33.00, 33.40 and 33.80 for 4.25, 4.35 and 4.45. With 600 litres, such an error is about Rs 120 for one farmer.

-**Module:** organisation (separate module, importable) remains appropriate.

C. Adaptation / Redesign (implemented)

Decision: modify, not rewrite. The function structure and module idea stay; the rate card becomes a dictionary per season selected by a season parameter, a separate bonus() function is added, Decimal replaces float, and steps are computed pro-rata.

Source: Task12/seasonal.py

```
"""Seasonal rate card + bonus. Fat/SNF readings may have 2 decimals; steps are pro-rata (fractional)."""
from decimal import Decimal as D, ROUND_HALF_UP
PAISE = D("0.01")
STD_FAT, STD_SNF, STEP = D("3.5"), D("8.5"), D("0.1")
SNF_STEP_VALUE = D("0.30")
RATE_CARDS = {
    "normal": {"base": D("30.00"), "fat_step": D("0.40")},
    "summer": {"base": D("32.00"), "fat_step": D("0.45")},   # April-June
}
BONUS_LIMIT, BONUS_PER_LITRE = D("500"), D("0.50")
SUMMER_MONTHS = (4, 5, 6)

def money(x): return x.quantize(PAISE, rounding=ROUND_HALF_UP)
def season_for_month(month): return "summer" if month in SUMMER_MONTHS else "normal"

def rate_per_litre(fat, snf, season="normal"):
    card = RATE_CARDS[season]
    return card["base"] + (fat - STD_FAT) / STEP * card["fat_step"] + (snf - STD_SNF) / STEP * SNF_STEP_VALUE

def payout(litres, fat, snf, season="normal"):
    return litres * rate_per_litre(fat, snf, season)

def bonus(total_litres):
    return total_litres * BONUS_PER_LITRE if total_litres > BONUS_LIMIT else D("0")

def ten_day_total(entries, season="normal"):
    total = sum((payout(l, f, s, season) for l, f, s in entries), D("0"))
    litres = sum((l for l, _, _ in entries), D("0"))
    return money(total + bonus(litres))

if __name__ == "__main__":
    print("rate 4.25/8.65 normal:", money(rate_per_litre(D("4.25"), D("8.65"))))
    print("rate 4.25/8.65 summer:", money(rate_per_litre(D("4.25"), D("8.65"), "summer")))
    small = [(D("10"), D("3.50"), D("8.50"))] * 10                    # 100 L
    big = [(D("60"), D("4.20"), D("8.70"))] * 10                      # 600 L
    assert ten_day_total(small, "normal") == D("3000.00")
    assert ten_day_total(small, "summer") == D("3200.00")             # 100 x 32
    assert ten_day_total(big, "normal") == money(600 * D("33.40") + 300)
    assert ten_day_total(big, "summer") == money(600 * (D("32.00") + 7*D("0.45") + 2*D("0.30")) + 300)
    assert bonus(D("500")) == 0 and bonus(D("500.01")) > 0            # strictly more than 500
    print("small normal/summer:", ten_day_total(small), ten_day_total(small, "summer"))
    print("big normal/summer :", ten_day_total(big), ten_day_total(big, "summer"))
    print("ALL SEASONAL TESTS PASSED")
```

D. Evidence-Based Defence

Test output (asserts: all pass)

```
rate 4.25/8.65 normal: 33.45
rate 4.25/8.65 summer: 35.83
small  normal/summer:  3000.00  3200.00
big  normal/summer  :  20340.00  21750.00
ALL SEASONAL TESTS PASSED
```

Case	Input (ten days)	Rate card	Bonus?	Expected total	Actual	Result
1	10 x (10 L, 3.50, 8.50) = 100 L	Original	No	100 x 30 = 3000.00	3000.00	PASS
2	same 100 L	Seasonal	No	100 x 32 = 3200.00	3200.00	PASS
3	10 x (60 L, 4.20, 8.70) = 600 L	Original	Yes (+300)	600 x 33.40 + 300 = 20340.00	20340.00	PASS
4	same 600 L	Seasonal	Yes (+300)	600 x 35.75 + 300 = 21750.00	21750.00	PASS
5	exactly 500 L / 500.01 L	-	No / Yes	bonus = 0 / > 0	as expected	PASS
6	fat 4.25, snf 8.65, 1 L	Original / Seasonal	-	33.45 / 35.83	33.45 / 35.83	PASS

(Case 4 rate: 32.00 + 7 x 0.45 + 2 x 0.30 = 35.75. Case 6 seasonal: 32.00 + 7.5 x 0.45 + 1.5 x 0.30 = 35.825, rounded half-up to 35.83.)

Trade-offs and conclusion

- **Retained:** function structure, module organisation, named-constant idea, testing approach.
- **Modified:** rate constants into a rate card keyed by season; Decimal arithmetic; fractional steps; bonus function with the ten-day total.

- **Cost:** Decimal is slower and more memory-hungry than float (Task 10), but the cost is milliseconds at 150,000 farmers and exactness is required for money. The season is a parameter, so tests for both seasons run without changing any code, and adding a new season is one dictionary entry.
- **Assumptions:** bonus applies toall litres once the total exceeds 500 (not only the excess) and exactly 500 L earns no bonus; the season is chosen from the collection month; fractional steps are paid pro-rata. If the co-operative decides otherwise, only one function or constant changes.

GitHub repository: https://github.com/swethasundar1108/CSA0801A